

Sebastian David Albu, BSc BSc

D-LAMP: Discrete LAplacian MPC Noise-Generating Protocols for Differentially Private Computation

**Implementing and Comparing Novel Noise Generation Protocols for the Discrete Laplacian
Distribution for Differentially Private Secure Multiparty Computation**

PROJECT

Master's degree programme: Software Engineering and Management

Supervisors

Dipl.-Ing. Fabian Schmid, BSc
Univ.-Prof. Dipl.-Ing. Dr.techn. Christian Rechberger

Institute of Information Security
Graz University of Technology

Graz, April 2025

Updated version from June 2025

Abstract

In today’s data-driven society, organizations such as finance and healthcare increasingly rely on advanced analytics and machine learning to extract actionable insights from sensitive data. However, traditional data aggregation methods pose serious privacy risks and often fail to meet stringent regulatory requirements. Secure multiparty computation (MPC) provides a promising alternative by enabling distributed parties to jointly compute functions without revealing their raw data. On the other hand, differential privacy (DP) ensures that the exact outputs do not leak individual information by perturbing the computed result with carefully calibrated noise. Combining MPC and DP thus offers a powerful approach to performing privacy-preserving data analysis and model training. This project focuses on an MPC noise-generation protocol that guarantee differential privacy using the Finite-range Discrete Laplacian Distribution (FDL). In particular, we present the first implementation of the FDL noise-generation protocol within the MP-SPDZ framework using SPDZ_{2^k} secret shares, complementing and extending earlier FDL implementations. Our approach uses a mixed-circuit design that interleaves arithmetic and binary operations. A key component is a domain-shift mechanism that converts numerical shares into binary representations using pre-distributed random bit pairs, and our findings show how one can avoid this complex computational overhead in these protocols. Our protocol significantly reduces round complexity and communication overhead through novel optimizations while maintaining strict DP guarantees even against active adversaries. Empirical evaluations demonstrate that our optimized implementation outperforms previous benchmarks in efficiency and achieves competitive accuracy for downstream tasks. Our work lays the groundwork for more scalable and robust privacy-preserving computations. Our improvements allow wider adoption of MPC-based differential privacy protocols in real-world applications, paving the way for secure collaborative analysis without compromising individual privacy.

Keywords: Differential Privacy · Secure Multiparty Computation · Noise Generation Protocols · Secret Sharing · Discrete Laplacian Distribution · MP-SPDZ

Contents

1	Introduction	4
2	Background	6
2.1	Differential Privacy (DP) and Noise Mechanisms	6
2.2	Secure Multi-Party Computation (MPC)	7
2.3	Secret Sharing	8
2.4	Differentially private MPC protocols	12
2.5	Finite-Range Discrete Laplace (FDL) Noise Generation	13
3	Finite-Range Discrete Laplacian (FDL1) Procedure	15
3.1	Constructing Finite-Range Discrete Laplace Noise	15
3.2	The procedure	16
3.3	Optimized Version of FDL1	20
4	Results	25
4.1	Performance vs. Number of Samples	25
4.2	Distribution of Noise Samples	26
5	Conclusion	28
	Bibliography	29

1 Introduction

In today’s data-driven world, as also stated in [EIKN22], many organizations such as banks [BTW12], hospitals [KHK+16], and large-scale survey agencies need to perform statistical analysis and machine learning on sensitive data [SSSS17]. However, privacy concerns and legal constraints prevent these parties from pooling their raw data into a central repository. *Secure multiparty computation* (MPC) offers a powerful solution by enabling parties to jointly compute a function over their private inputs while revealing nothing more than the final output [EKR+18].

While MPC protocols yield exact computation results, the leakage that occurs through these exact outputs may compromise individual privacy [SSSS17]. To mitigate this, *differentially private* (DP) mechanisms are employed. Differential privacy perturbs the exact results by adding carefully calibrated noise, thereby ensuring that outputs on similar inputs are statistically indistinguishable [Dwo06; DKM+06]. The combination of MPC and DP is a promising approach to privately compute useful functions over distributed data without sacrificing the formal privacy guarantees of both worlds [MRS24; EIKN22].

Since parties’ inputs are sensitive, it is not appropriate to assume a trusted party who aggregates their data and applies a mechanism to them. Instead, the mechanism must be realized in a fully distributed setting to emulate such a trusted party. Generally, given an MPC protocol generating noise, one can emulate a noise-adding mechanism and obtain a protocol that guarantees the same level of differential privacy [EIKN22; BNO08]. However, it is not straightforward to efficiently realize distributed noise generation. This is mainly because the noise must be sampled from a non-uniform distribution in a distributed manner even when corrupted parties may control their local randomness. In particular, a simple protocol where a designated party samples and shares noise, or where each party independently generates non-uniform noise r_i to perturb an outcome by adding $\sum_{i \in [n]} r_i$, does not work since it fails to prevent leakage or manipulation by adversarial parties [EIKN22].

In prior work [MRS24; EIKN22], protocols for the *Finite-Range Discrete Laplace Distribution* (FDL) labeled FDL1 and FDL2 were introduced in [EIKN22], where both were shown to satisfy differential privacy in a secure MPC setting. The practical applicability of FDL2 in secure MPC was later demonstrated in [MRS24] through its implementation in the MP-SPDZ framework [MP-25]. Our work focuses on implementing the FDL1 noise-generation protocol in MP-SPDZ. In doing so, we bridge the gap between FDL1 and FDL2 implementations and significantly optimize FDL1 regarding communication complexity and overall runtime.

The interactions in Multi-Party Computation (MPC) involve multiple parties, ensuring that no sensitive information is leaked through the values obtained. *Secret shares* in MPC are parts of a secret distributed among the parties, such that no single party can learn the secret without collaborating with others. These shares can be either $\langle r \rangle$, which are arithmetic shares (in $\mathbb{Z}_{2^k}$), or $[r]$, which are binary shares (in $\mathbb{Z}_2$). Our method is designed to interleave operations on whole numbers with bit-level manipulations. At its core, it features a procedure that converts a

distributed numerical share ($\langle r \rangle$) into its binary form ($[r]$) by masking it with pre-distributed random bit pairs. The same holds vice-versa when we convert from binary shares to arithmetic shares. These conversions (or domain shifts) are critical, as the resulting binary representation is then used to generate the noise required for differential privacy and overcome limitations of the corresponding domains, as not all operations on secret shares are possible in either domain. Our extensive experiments show significant performance improvements when we avoid switching the computation domains of our shares and how we can avoid these costly operations and stay inside the binary domain for most of our protocols.

The main scientific contributions of our project are as follows:

- We present the first implementation of the FDL1 noise-generation protocol from [EIKN22] in the MP-SPDZ framework [MP-25], complementing the earlier FDL2 implementations [MRS24].
- We design a protocol that bridges whole-number (arithmetic) operations and bit-level operations by converting numerical values to binary form using pre-distributed random bit pairs and vice-versa.
- We rigorously analyze our optimizations and demonstrate that our protocol satisfies differential privacy guarantees even against active adversaries.
- Extensive empirical evaluations show that our optimized FDL1 implementation outperforms previous benchmarks while achieving competitive accuracy.

In summary, our work advances the secure noise generation for differential privacy in MPC. By optimizing the FDL1 protocol from [EIKN22] for the MP-SPDZ [MP-25] environment, where we use state-of-the-art type of secret shares (SPDZ_{2^k} [CDE+18]), we achieve a practical balance between strong theoretical guarantees and efficient implementation.

Outline. The remainder of this thesis is organized as follows. In chapter 2, we review the fundamental concepts of secure multiparty computation and differential privacy as well as some fundamentals about secret sharing and our procedures that allow domain transformations for secret shares with the help of some preprocessed bits. We also briefly summarize why the protocol proposed by [EIKN22] provides DP, and explain how adding carefully designed noise to a sensitive function can guarantee differential privacy. Afterward, in chapter 3 the *Finite-range Discrete Laplace* (FDL1) procedure, from [EIKN22], is explained, as well as the optimization of the procedure. Finally, chapter 4 presents extensive experimental results, and we conclude with a discussion of future research directions in chapter 5.

2 Background

This chapter introduces the fundamental terminology of all concepts used in this master project, beginning with the definitions of *Differential Privacy* (DP) and *Secure Multiparty Computation* (MPC), as well as the concept of *Secret Sharing* (SS) and how the domain shifts between *arithmetic shares* and *binary shares* work.

Differential Privacy (DP). Introduced by [Dwo06], DP has emerged as a cornerstone of privacy-preserving data analysis. Simplified, it ensures that the output of a computation does not reveal significant information about any individual input, even to an adversary with auxiliary knowledge. The core idea behind DP is to add noise to the results of computations so that the presence or absence of any individual’s data does not substantially affect the outcome, thus ensuring privacy.

Multi-Party Computation (MPC). Introduced by [Yao82], with the first practical application documented in [BCD+09], MPC, on the other hand, allows multiple parties to collaboratively compute a function over their joint data while keeping each party’s input private. Each party inputs its data to the computation, but no party learns anything about the inputs of others beyond what can be inferred from the final result. This is achieved through cryptographic techniques, enabling secure computation even in the presence of malicious adversaries [EKR+18].

Combining MPC and DP. The combination of Differential Privacy and MPC is compelling and has been first proposed in [DKM+06]. While DP ensures that individual data points are protected through noise addition, MPC allows parties to perform computations on sensitive data without revealing the data itself. This synergy enables privacy-preserving computations that provide robust security guarantees for both individual privacy and the confidentiality of data inputs, even in the presence of multiple collaborating parties or adversaries. Together, these techniques can enable secure, privacy-preserving data analysis in environments where individual privacy and collaborative computation are crucial [MRS24].

2.1 Differential Privacy (DP) and Noise Mechanisms

Differential privacy is formally defined as follows [Dwo06]:

(ϵ, δ) -Differential Privacy. A randomized mechanism $M : \mathcal{D} \rightarrow \mathcal{R}$ satisfies (ϵ, δ) -differential privacy if for all adjacent datasets $D, D' \in \mathcal{D}$ differing in at most one entry and for all measurable subsets $S \subseteq \mathcal{R}$:

$$\Pr[M(D) \in S] \leq e^\epsilon \Pr[M(D') \in S] + \delta,$$

where $\epsilon > 0$ controls the privacy loss and $\delta \geq 0$ allows a small probability of failure.

A common approach to achieve differential privacy is through additive noise mechanisms, where controlled random noise is added to the query results. Prominent examples include:

- **Laplace Mechanism:** Introduced by [Dwo06], alternatively in Definition 3.3 from [DR14]. Adds noise drawn from the Laplace distribution, calibrated to the ℓ_1 -sensitivity (see Theorem 3.1 from [DR14]) of the query function. In mathematical terms, a minimal noise level is adequate for a weak privacy constraint, which aligns with a larger value of ϵ . Conversely, an increased noise level enhances the uncertainty regarding the original input, corresponding to a smaller value of ϵ .
- **Discrete Laplace Mechanism:** Also known as the *Geometric Mechanism*, it injects integer noise drawn from a two-sided geometric distribution to achieve $(\epsilon, 0)$ -differential privacy in MPC settings [EIKN22].
 - **Privacy parameter $\epsilon > 0$:** Controls the strength of privacy. Smaller ϵ means stronger privacy (and more noise), larger ϵ means less noise and weaker privacy.
 - **Sensitivity $\Delta \geq 1$:** The maximum change in the query’s integer output when altering any single input record. Higher Δ requires more noise to mask that change.

Concretely, set

$$p = \exp\left(-\frac{\epsilon}{\Delta}\right),$$

and sample $L \sim \text{DL}(p)$ with

$$\Pr[L = k] = p^{|k|} \frac{1-p}{1+p}, \quad k \in \mathbb{Z}.$$

This yields an integer-valued mechanism whose noise decays exponentially with $|k|$, preserves $(\epsilon, 0)$ -privacy, and is known to be minimax-optimal under any symmetric, monotone loss [GRS08; EIKN22].

The discrete Laplace mechanism can be used to add noise to the output of a function $g(x)$, where the added noise L is drawn from $\text{DL}(p)$, ensuring that the resulting output satisfies $(\epsilon, 0)$ -differential privacy. In this case, the added noise ensures that the probability of an adversary distinguishing between two neighboring inputs is bounded by $\exp(\epsilon)$.

For practical purposes, the discrete Laplace distribution is often truncated to a finite range to ensure that the added noise is bounded, especially when working with integer-valued data. This finite-range version still preserves the differential privacy guarantees, as proven in [EIKN22] while ensuring the noise is within a reasonable range for practical applications.

2.2 Secure Multi-Party Computation (MPC)

Secure multi-party computation enables multiple parties to jointly compute a function over their inputs while keeping them private. MPC protocols often rely on secret sharing or garbled circuits [EKR+18]. In differential privacy, noise generation protocols in MPC are crucial to emulate a trusted party while avoiding a central dataholder. We can define the secure MPC setting from [EIKN22].

Secure MPC setting. We consider n parties, each holding a private input $x_i \in D$ for some finite domain D . Let $g: D^n \rightarrow R$ be the deterministic functionality to be computed, and let Π denote an MPC protocol for g . We fix a threshold $t < n$ and assume a static adversary that

may corrupt any subset $T \subseteq [n]$ of up to t parties (i.e. a t -party collusion model). Unless stated otherwise, we aim for *computational* security¹ against *passive* (semi-honest) adversaries². For any execution of Π on inputs $\mathbf{x} = (x_1, \dots, x_n)$, define the *view* of parties in T as

$$\text{View}_{\Pi}^T(\mathbf{x}) = (x_T, \rho_T, \text{all messages sent and received by parties in } T),$$

where ρ_T denotes their local randomness. We say that Π is a t -secure MPC protocol for g if there exists a simulator $\mathcal{S}_T$ such that, for every T of size $\leq t$ and all input vectors $\mathbf{x}, \mathbf{y}$ with

$$x_T = y_T \quad \text{and} \quad g(\mathbf{x}) = g(\mathbf{y}),$$

the simulated view $\mathcal{S}_T(x_T, g(\mathbf{x}))$ is *indistinguishable* from the real view $\text{View}_{\Pi}^T(\mathbf{x})$. Moreover, correctness requires that each honest party learns the output:

$$g(\mathbf{x}) \in \text{View}_{\Pi}^{\{i\}}(\mathbf{x}) \quad \text{for all } i \in [n].$$

Takeaway: This adapted definition from [EIKN22] guarantees that any coalition of up to t passive corruptions learns nothing beyond their inputs and the final output, ensuring both privacy and correctness under our chosen adversarial and security models.

2.3 Secret Sharing

From [EKR+18], we can define the concept of *Secret sharing*, which is a key cryptographic primitive that forms the foundation of many secure MPC protocols. In a (t, n) -secret sharing scheme, a secret s is divided into n shares, with the following properties:

- Any subset of fewer than t shares reveals no information about the secret s . Depending on the sharing scheme used, it can also hold that $t = n$.
- Any collection of exactly t shares allows the complete reconstruction of the secret s .

Various forms of secret sharing exist, each with different security guarantees. Below, we present a definition of a secret sharing scheme, adapted from [BC92], which captures the key properties required for secure sharing and reconstruction of a secret.

Definition: Secret Sharing Scheme. Let D represent the domain of the secret and D_1 the domain of the shares. A secret sharing scheme is characterized by two algorithms: $\text{Shr} : D \rightarrow D_1^n$, which is a (potentially randomized) sharing algorithm, and $\text{Rec} : D_1^k \rightarrow D$, the reconstruction algorithm. The pair of algorithms (Shr, Rec) defines a (t, n) -secret sharing scheme and it must satisfy two essential properties:

1. **Correctness.** Given a secret s , the algorithm $\text{Shr}(s)$ produces the shares $(s_1, s_2, \dots, s_n)$. The reconstruction algorithm Rec guarantees that, when at least t shares are provided, the original secret is correctly reconstructed:

$$\Pr [\forall k \geq t, \text{Rec}(s_{i_1}, \dots, s_{i_k}) = s] = 1.$$

¹Computational security guarantees that an adversary with bounded computational resources (e.g., probabilistic polynomial-time algorithms) cannot break the protocol except with negligible probability. This security relies on computational hardness assumptions (e.g., factoring large integers, discrete logarithms) [GMW87]

²A passive adversary follows the protocol correctly but attempts to infer private information from the transcript of messages exchanged during execution. Also termed ‘honest-but-curious’ [GMW87]

2. **Perfect Privacy.** Any set of shares smaller than t provides no information about the secret. In other words, for any two secrets $a, b \in D$, and any set of $k < t$ shares $v = (v_1, v_2, \dots, v_k)$, the probability distribution over shares is identical for both secrets:

$$\Pr[v = \text{Shr}(a)|k] = \Pr[v = \text{Shr}(b)|k],$$

where $|k$ indicates the projection onto a subspace of k shares.

SPDZ_{2^k} Shares. In our implementation, we use the *SPDZ_{2^k} secret-sharing scheme* introduced by [CDE+18]. This scheme operates in the arithmetic domain over rings $\mathbb{Z}_{2^k}$ (for $k \geq 1$) and is designed for *multi-party dishonest-majority*³ settings. Unlike schemes requiring large prime fields, SPDZ_{2^k} efficiently handles arithmetic modulo 2^k using *additive secret sharing* (ASS) secured by information-theoretic *message authentication codes* (MACs)⁴ over an extended ring $\mathbb{Z}_{2^{k+\kappa}}$. For preprocessing, it employs *Oblivious Transfer (OT)*⁵ rather than homomorphic encryption (HE)⁶ to generate multiplication triples, as noted in [CDE+18]:

“SPDZ_{2^k} extends the *OT-based approach* proposed in MASCOT [KOS16] to the $\mathbb{Z}_{2^{k+\kappa}}$ setting.”

This OT-centric design avoids costly HE operations while maintaining active security, making it suitable for practical applications like fixed-point arithmetic, on which we also rely in the noise sampling protocol.

Arithmetic vs. Binary Secret Sharing. In secure MPC, a value r can be secret-shared either in the *arithmetic domain*, denoted $\langle r \rangle$, or in the *binary domain*, denoted $[r]$. An arithmetic share $\langle r \rangle$ represents additive shares of r over a ring $\mathbb{Z}_p$, and is ideal for performing additions and multiplications directly on shared values (e.g. inner products, fixed-point arithmetic). In contrast, a binary share $[r]$ splits each bit of r across parties modulo 2, enabling efficient evaluation of Boolean primitives such as comparisons, bit-wise operations, and conditional gates. Protocols often convert between the two representations, using, for instance, pre-distributed random bit pairs (da-Bits) and conversion routines, so as to leverage the arithmetic shares for heavy numeric work and the binary shares for control-flow or bit-level subroutines.

Notation for Secret Share Vectors. For a vector of shares $y = (y_1, \dots, y_n)$, we write

$$\|y\|_1 = \sum_{i=1}^n |y_i|,$$

where $|y_i|$ denotes the usual absolute value of the integer or field-element y_i .

³In secure multi-party computation (MPC), a dishonest-majority setting refers to scenarios where more than half of the participating parties may be corrupted. Despite this, protocols can still ensure correctness and privacy, even against active (malicious) adversaries. This contrasts with honest-majority settings, which require a majority of participants to be trustworthy [CDD+98].

⁴Message Authentication Codes (MACs) verify message integrity and authenticity using symmetric keys, ensuring data originates from trusted sources and remains unaltered. [Nat85].

⁵Oblivious Transfer (OT) is a fundamental cryptographic protocol wherein a sender possesses multiple pieces of information, and a receiver retrieves one piece without revealing which one was chosen, while the sender remains unaware of the selection. In the MASCOT protocol, OT is utilized to securely generate multiplication triples during the preprocessing phase, enabling efficient and actively secure multi-party computation without relying on costly homomorphic encryption techniques [KOS16].

⁶Homomorphic encryption (HE) allows computations on ciphertexts, producing encrypted outputs that, when decrypted, yield the same result as if operations were applied to the plaintext, first realized by [Gen09]

da-Bits (Double authenticated Bits). We briefly discuss *da-Bits* (double-authenticated bits) [DEK21], which are a common technique used under the hood to make the process of domain conversions/transformations efficient and actively secure and which are also commonly used in the framework of MPSDZ [MP-25], which we use for the implementation of our procedures. A *da-Bit* is a preprocessed pair $(\langle r \rangle, [r])$ representing the *same random bit* r in two different domains:

- $\langle r \rangle$ is a *secret share in the arithmetic domain* (e.g., $\mathbb{Z}_{2^k}$).
- $[r]$ is a *secret share in the binary domain* (mod 2).

These pairs are generated offline with active security checks, ensuring the bit is consistent in both domains.

Domain Transformations. Our protocol also needs to perform domain transformations, therefore, we discuss the reasons for domain transformation, and our transformation algorithms. To perform an integer summation (and multiplications, subtractions, etc.), binary shares must be *converted* or *interpreted* in the arithmetic domain. In many MPC frameworks, binary shares (e.g., mod 2) are incompatible with typical integer or field operations (e.g., mod p or mod 2^k). A standard technique is used to securely convert each binary share into an arithmetic share of the same bit (often called “binary-to-arithmetic conversion” or “bit-decomposition” and “recomposition”). Transforming from one domain to another takes significant effort, especially regarding interactions between parties in a secure MPC setting. We also discuss this aspect in more detail later when we compare the different implementation methods of our MPC noise sampling procedure in chapter 3. In Figure 2.1 we demonstrate a *generic procedure* that explains how to *convert binary shares* $([b_i])$ into *arithmetic shares* $(\langle b_i \rangle)$ within an secure MPC framework. This is often referred to as *binary-to-arithmetic conversion*.

Since we are in a secure MPC setting, we cannot simply open shared values and need to mask those values first. For each binary share $[b_i]$ we want to convert, we XOR it with the binary portion of a da-Bit, $[r_i]$. This “masked” bit $[b_i \oplus r_i]$ is then opened (declassified) to all parties. The revealed value $m_i = b_i \oplus r_i$ does *not* directly leak b_i if r_i is random and unknown to any one party individually. The da-Bits are particularly useful here since each da-Bit also has an arithmetic component $\langle r_i \rangle$. We can correct it based on the revealed m_i . A small arithmetic trick yields $\langle b_i \rangle$. Essentially, if $m_i = 0$, the arithmetic share is $\langle r_i \rangle$. If $m_i = 1$, we flip that share to get the correct bit in the arithmetic domain. At the end, we have $\langle b_i \rangle$ for each bit b_i . We can now sum or multiply these $\langle b_i \rangle$ shares like regular integers in $\mathbb{Z}_p$ or $\mathbb{Z}_{2^k}$.

The transformation from the *arithmetic domain* to the *binary domain* follows a similar principle. We outline the procedure in Figure 2.2.

The transformation from the arithmetic to the binary domain begins by taking an arithmetic share $\langle x \rangle$, a target bit-length L , and a precomputed set of L da-Bit pairs $(\langle r_i \rangle, [r_i])$. First, one constructs an arithmetic mask

$$r = \sum_{i=0}^{L-1} 2^i \langle r_i \rangle$$

and collects the corresponding binary mask shares into a vector r_{bin} . Next, the protocol computes the masked share

$$\langle y \rangle = \langle x \rangle + \langle r \rangle$$

and opens $\langle y \rangle$ to all parties. Since r is uniformly random, this reveals no information about $\langle x \rangle$. Once the opened value y_{open} is available, it is converted into its binary expansion y_{bin} . Finally,

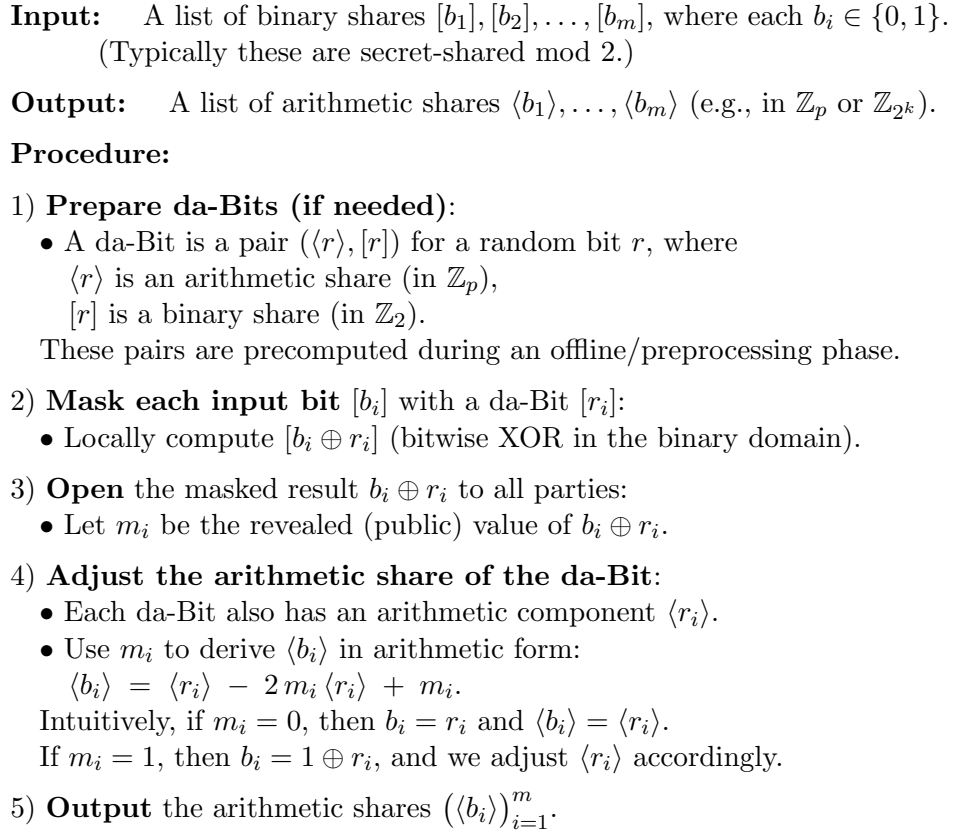


Figure 2.1: Binary-to-Arithmetic Conversion using da-Bits.

each bit-share $[b_i]$ of the original x is recovered via secure binary subtraction,

$$[b_i] = y_{\text{bin}} - [r_i],$$

using the *binary subtraction circuit* detailed in Figures 3.7 and 3.9 (see also chapter 3). The output is thus a vector of binary shares whose reconstruction yields the original value x .

Secure Boolean Operations. The following boolean operations are needed for the previously mentioned subtraction circuit in the binary domain. All Boolean gates on secret-shared bits are built from two primitives in $\text{GF}(2)$:

$$\text{secure_xor}(a, b) = a + b \quad \text{and} \quad \text{secure_and}(a, b) = \text{secure-multiplication}(a, b).$$

From these we get

$$\text{secure_or}(a, b) = \text{secure_xor}(a, b) + \text{secure_and}(a, b), \quad \text{secure_not}(a) = 1 - a.$$

In chapter 3, we combine these primitives into a standard full-subtractor circuit (over just the c LSBs) plus a sign-extension step to realize two's-complement subtraction. The detailed gate-by-gate description is deferred to Figure 3.7.

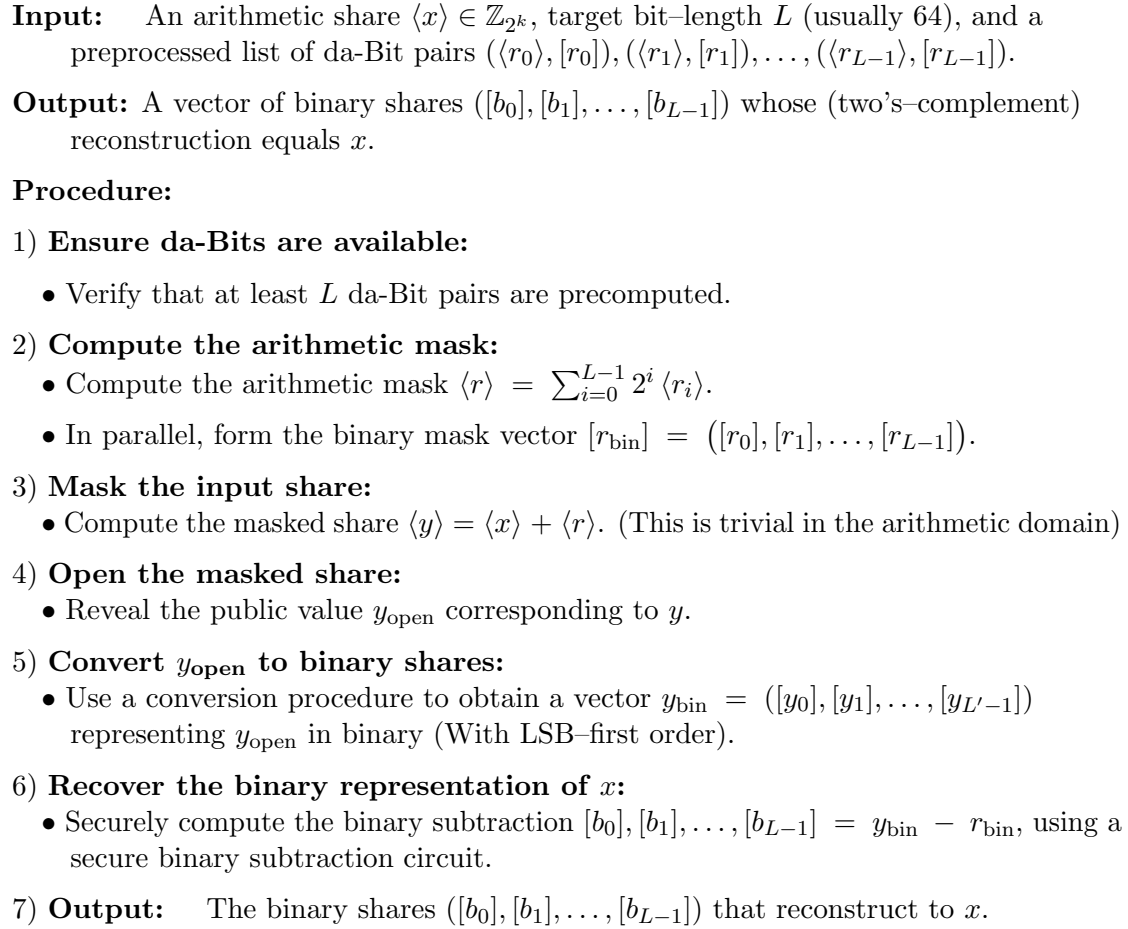


Figure 2.2: Arithmetic-to-Binary Conversion using da-Bits.

2.4 Differentially private MPC protocols

We now aim to combine both principles of MPC and DP. From [EIKN22], we can define a differentially private MPC protocol as follows: An MPC protocol Π is said to be a $(t; \epsilon, \delta)$ -differentially private protocol for computing a function g if the following conditions are satisfied:

- **t-Privacy:** For any subset T of size at most t , and for any pair of inputs x and y such that $x_T = y_T$ and $g(x) = g(y)$, the distributions of the views of the protocol on T , denoted by $\text{View}_{\Pi, T}(x)$ and $\text{View}_{\Pi, T}(y)$, must be identical.
- **(ϵ, δ) -Differential Privacy:** For any subset T of size at most t , and for any pair of T -neighboring inputs x and y , the distributions of $\text{View}_{\Pi, T}(x)$ and $\text{View}_{\Pi, T}(y)$ should be (ϵ, δ) -differentially private, meaning the distributions are close within the bounds defined by ϵ and δ .

2.5 Finite-Range Discrete Laplace (FDL) Noise Generation

Let $g : \mathcal{D}^n \rightarrow \mathbb{Z}$ be a *sensitive integer-valued query* (e.g., a count or sum over datasets). Its *sensitivity* Δ is defined as:

$$\Delta = \max_{\substack{x, x' \in \mathcal{D}^n \\ \text{neighbouring}}} |g(x) - g(x')|,$$

where datasets x, x' are *neighbouring* if they differ in exactly one record. To achieve differential privacy, we perturb $g(x)$ with finite-range discrete Laplace noise:

$$M(x) = g(x) + X, \quad X \sim \text{FDL}(p, N), \quad p = e^{-\epsilon/\Delta}.$$

Here $\text{FDL}(p, N)$ is the finite-range discrete Laplace distribution [EIKN22] supported on $\{-N + 1, \dots, N - 1\}$. The parameter N is chosen as the smallest integer satisfying:

$$\Pr[|X| \geq N - \Delta] \leq \delta.$$

By [EIKN22], M satisfies (ϵ, δ) -DP if X has:

1. **Bounded Support:** $X \in \{-N + 1, \dots, N - 1\}$
2. **Symmetry:** $\Pr[X = k] = \Pr[X = -k] \forall k \geq 0$
3. **Exponential Decay:** $\forall 0 \leq z \leq z' \leq z + \Delta$:

$$\Pr[X = z] \leq e^\epsilon \cdot \Pr[X = z']$$

Proof of (ϵ, δ) -DP . The following is a slightly adapted and shortened proof from [EIKN22]. Let x, x' be neighbouring datasets with $|g(x) - g(x')| \leq \Delta$. Assume w.l.o.g. $g(x') \leq g(x)$. For any output set $S \subseteq \mathbb{Z}$, partition:

$$S = S_1 \cup S_2, \quad \text{where} \quad \begin{aligned} S_1 &= S \cap [g(x) - N, g(x') + N], \\ S_2 &= S \setminus S_1. \end{aligned}$$

Note that $g(x) - N \leq g(x') + N$ holds because $|g(x) - g(x')| \leq \Delta \leq N$ (since $N \geq \Delta$ by the tail bound).

- **Tail bound (S_2):** For $z \in S_2$:

- If $z \leq g(x) - N - 1$, then $z - g(x) \leq -N - 1 < -N + 1$. Impossible since $X \geq -N + 1$.
- If $z \geq g(x') + N + 1$, then:

$$z - g(x) \geq (g(x') + N + 1) - g(x) \geq (g(x) - \Delta + N + 1) - g(x) = N + 1 - \Delta.$$

Thus $|X| \geq N + 1 - \Delta \geq N - \Delta$ (since $N + 1 - \Delta \geq N - \Delta$). By the choice of N :

$$\Pr[M(x) \in S_2] \leq \Pr[|X| \geq N - \Delta] \leq \delta.$$

- **Exponential decay (S_1):** For $z \in S_1$, define the noise values:

$$X_z^x = z - g(x), \quad X_z^{x'} = z - g(x').$$

The difference is bounded by sensitivity:

$$|X_z^x - X_z^{x'}| = |g(x') - g(x)| \leq \Delta.$$

Since $0 \leq X_z^x \leq X_z^{x'} \leq X_z^x + \Delta$ (because $g(x') \leq g(x)$), the *Exponential Decay* property implies:

$$\Pr[X = X_z^x] \leq e^\epsilon \cdot \Pr[X = X_z^{x'}].$$

Summing over $z \in S_1$:

$$\Pr[M(x) \in S_1] \leq e^\epsilon \sum_{z \in S_1} \Pr[M(x') = z] \leq e^\epsilon \Pr[M(x') \in S].$$

Combining both cases:

$$\Pr[M(x) \in S] \leq e^\epsilon \Pr[M(x') \in S] + \delta.$$

This holds for all neighbouring x, x' and measurable S , proving (ϵ, δ) -DP.

3 Finite-Range Discrete Laplacian (FDL1) Procedure

This chapter presents the *Finite-Range Discrete Laplacian Procedure* [EIKN22], a constant-round protocol for generating noise in differential privacy, abbreviated with FDL1, as seen in [EIKN22]. The construction proceeds in two main steps. First, we build a Bernoulli noise generator from uniformly random bits. Then, we use this as a building block to construct a finite-range variant of the discrete Laplace distribution using two Geometric distributions of those Bernoulli samples. The difference between those two is then an approximation of $DL(p)$, as backed up by [IK06]. The procedure for FDL1 has first been proposed and asymptotically analyzed in [EIKN22].

3.1 Constructing Finite-Range Discrete Laplace Noise

In MPC-based DP the key is to generate noise without ever revealing individual shares, so [EIKN22] builds the discrete Laplace by first drawing two independent truncated geometric variables and subtracting them. Concretely, the procedure never samples the geometric variable in one go, but instead runs a small sequence of Bernoulli trials whose success probabilities β_i are defined¹ as

$$\beta_i = \frac{1}{1 + p^{-2^i}} \quad (i = 0, 1, \dots, \lfloor \log_2(N) \rfloor)$$

fall off exponentially in i . This “binary decomposition” ensures that the chance of a large geometric outcome drops rapidly, concentrating most of the mass near zero. The resulting distribution on $\{-N + 1, \dots, N - 1\}$, denoted $FDL_1(p, N)$, resembles the discrete Laplace distribution. For a concrete proof as to why this results in the finite-range discrete Laplace distribution and why it is therefore enforcing DP, we refer to [IK06; EIKN22]. A simplified proof for the finite-range discrete Laplace distribution enforcing DP can be found in section 2.5. In applications one often further conditions X to lie in $[-M, M]$ for some $M \in \mathbb{Z}$ with $M \leq N - 1$, yielding the practical variant $FDL_1(p, N, M)$ [EIKN22]. In [EIKN22] there is no explicit bound stated for M . However, the probability of failure is given, which in practice could be mapped to a potential range for M . This could be implemented in future publications and would just need minor dynamic adjustments, as our range check comparisons (Figure 3.10, Figure 3.11) work $\forall M \in \mathbb{N}$, s.t. $M < N$, as given in [EIKN22]. The probability of failure (which could be a possible dynamic bound for M) given by [EIKN22], is

$$\frac{2p^{M+1}}{(1+p)(1-p^N)^2}.$$

¹It appears there’s a small typo in [EIKN22]: they write $\beta_i = (1 + p^{2^{-i}})^{-1}$, but we will assume they meant $\beta_i = (1 + p^{-2^i})^{-1}$.

3.2 The procedure

The steps involved in Figure 3.1 describe the FDL1 procedure, as described in [EIKN22]. We now aim to discuss all steps in this section, starting with the input of the FDL1 procedure.

Input of Procedure. We assume, as input, a collection

$$\left([a_{i1}], \dots, [a_{id}]\right)_{i \in [0..c)} \quad \text{and} \quad \left([b_{i1}], \dots, [b_{id}]\right)_{i \in [0..c)},$$

where $a_{ij}, b_{ij} \in \{0, 1\}$. These represent c vectors of d independent simulated coin flips. Let $c, N, M \in \mathbb{N}$, whereas it holds that $N = 2^c$, $M < N$, and $\beta_i = (1 + p^{-2^i})^{-1}$ for $i \in [0..c)$, as given in [EIKN22]².

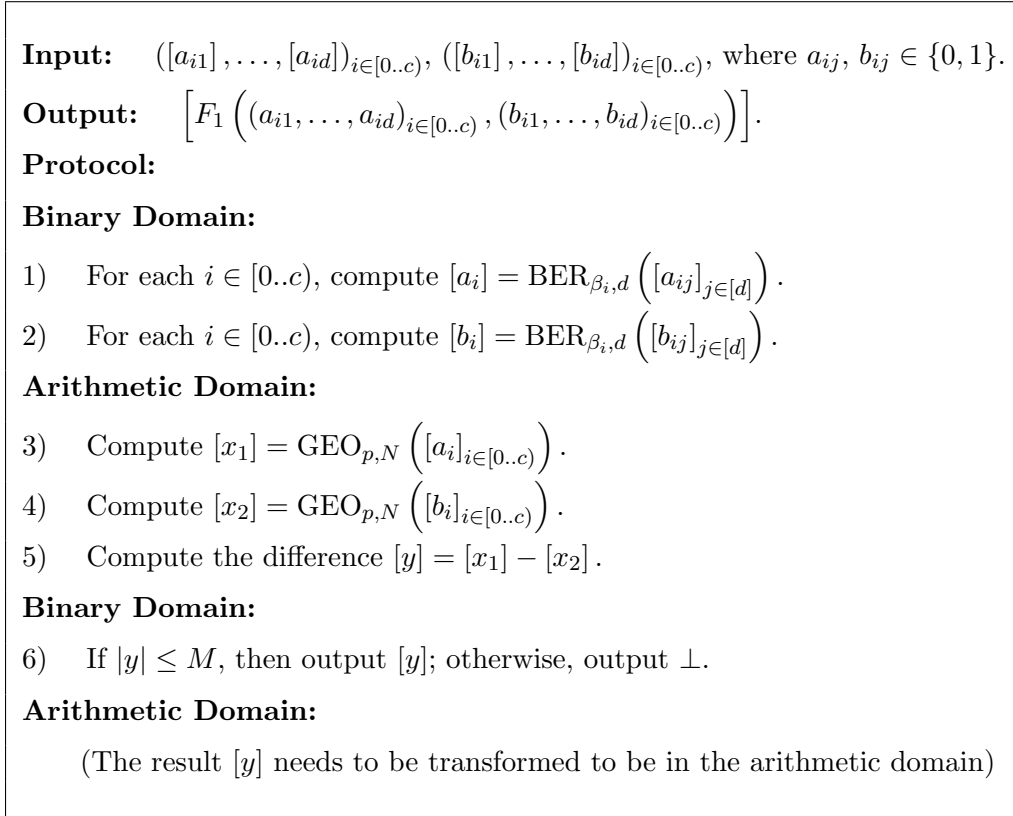


Figure 3.1: FDL1 Procedure for computing $F_1 \left((a_{i1}, \dots, a_{id})_{i \in [0..c)}, (b_{i1}, \dots, b_{id})_{i \in [0..c)} \right)$, where the GEO function generates geometric noise.

1. **Approximate Bernoulli Procedure (BER).** In the first two steps, an approximation of a Bernoulli distribution is computed, as outlined in Figure 3.2. This procedure uses a d_{prob} -bit (LSB-first) representation of a probability β along with a seeded PRNG to generate c approximate Bernoulli samples entirely in the binary domain. Each $[b_i]$ is computed from the binary representations of $([b_{ij}])_{j \in [d]}$. This step operates in the binary domain and ensures that the values conform to the desired Bernoulli distribution. A

²Adapted values for β_i , as discussed previously

high-level view: given d bits $\{b_\ell\}$ (each $b_\ell \in \{0, 1\}$) and corresponding bias bits $\{\beta_\ell\}$, the protocol produces a single bit output that follows a Bernoulli distribution with some parameter derived from $\{\beta_\ell\}$. Internally, it uses two subroutines (taken from [EIKN22]):

- A **Prefix OR** function $\text{PRE}_d^\vee$, summarized in Figure 3.4.
- An **Inner Product** function IP^d , summarized in Figure 3.3.

The XOR with β_ℓ toggles each bit b_ℓ if needed. The prefix OR then identifies the *first* position where a one appears (if any). By taking the differences of the prefix OR outputs, we isolate that first-1 position. Finally, an inner product with the original bits identifies whether that position corresponds to a “success” or not, and the final output is flipped accordingly by taking one and subtracting the previous outcome. This simulates a regular Bernoulli distribution, in which it holds that

$$\mathbb{P}(X = x) = \begin{cases} p & \text{if } x = 1 \\ 1 - p & \text{if } x = 0 \end{cases}$$

where:

- p is the probability of success (i.e., $\mathbb{P}(X = 1) = p$),
- $1 - p$ is the probability of failure (i.e., $\mathbb{P}(X = 0) = 1 - p$),
- $0 \leq p \leq 1$.

The procedure is summarized in Figure 3.2.

Input: $(b_\ell)_{\ell \in [d]}$, where each $b_\ell \in \{0, 1\}$.
Bias bits $(\alpha_\ell)_{\ell \in [d]}$.

Output: $B_{\alpha, d}(b_1, \dots, b_d)$.

Protocol:

- 1) $c_\ell = b_\ell \oplus \alpha_\ell$ for each $\ell \in [d]$.
- 2) $(e_\ell)_{\ell \in [d]} = \text{PRE}_d^\vee((c_k)_{k \in [d]})$.
- 3) $f_\ell = e_\ell - e_{\ell-1}$ for $\ell \in [d]$, with $e_0 = 0$.
- 4) $g = \text{IP}^d((f_\ell)_{\ell \in [d]}, (b_\ell)_{\ell \in [d]})$.
- 5) **Output:** $1 - g$.

Figure 3.2: Bernoulli procedure $\text{BER}_{\alpha, d}$ for computing $B_{\alpha, d}(b_1, \dots, b_d)$.

In the first step of Figure 3.2 we XOR with Bias Bits, meaning $(c_\ell = b_\ell \oplus \alpha_\ell)$. This “mixes” each input bit b_ℓ with a bias bit α_ℓ . Conceptually, this can flip b_ℓ if the procedure requires it for the correct Bernoulli distribution. In the second step we compute the Prefix OR, $\text{PRE}_d^\vee((c_k)_{k \in [d]})$. This sub-procedure is also summarized in Figure 3.4. Using the subroutine $\text{PRE}_d^\vee$, we compute cumulative ORs. Intuitively,

$$e_\ell = (c_1 \vee c_2 \vee \dots \vee c_\ell),$$

indicating whether at least one “1” has appeared up to index ℓ . In the third step of $\text{BER}_{\alpha,d}$, we compute differences ($f_\ell = e_\ell - e_{\ell-1}$). These differences pick out exactly the transition point from 0 to 1 in the prefix OR sequence (if it exists). The difference is one only at the first location where a “1” appears; otherwise, it is 0. In step 4 we compute the inner product ($g = \text{IP}^d((f_\ell), (b_\ell))$). This sub-procedure is also summarized in Figure 3.3. The procedure IP^d sums up the products $f_\ell \cdot b_\ell$. Because f_ℓ isolates the index of the first “1”, this inner product effectively checks whether that first “1” truly corresponds to a “success” in the original b_ℓ . The final output is now calculated with $1 - g$. The protocol flips the bit g to align the final output with the intended Bernoulli sampling. (Sometimes, the internal logic yields a zero if success occurs, so the final flip ensures the correct sign convention.)

Input: $(x_\ell)_{\ell \in [d]}, (y_\ell)_{\ell \in [d]}$.

Output: $\text{IP}^d((x_\ell), (y_\ell)) = \sum_{\ell=1}^d x_\ell \cdot y_\ell$.

Procedure:

- 1) For $\ell = 1$ to d , compute $z_\ell = x_\ell \times y_\ell$.
- 2) Compute $z = \sum_{\ell=1}^d z_\ell$.
- 3) Output z .

Figure 3.3: Inner Product subroutine IP^d .

This subroutine IP^d computes the dot product of two d -element vectors over some ring (often integers or a field). For bits, it’s simply:

$$\text{IP}^d((x_\ell)_{\ell=1}^d, (y_\ell)_{\ell=1}^d) = \sum_{\ell=1}^d (x_\ell \times y_\ell).$$

We take two vectors $(x_\ell)_{\ell \in [d]}, (y_\ell)_{\ell \in [d]}$ as an input, and the procedure outputs a single integer/bit representing $\sum_{\ell=1}^d x_\ell \cdot y_\ell$. In an MPC protocol, each x_ℓ and y_ℓ might be secret-shared. Then IP^d is typically computed by first multiplying each pair (x_ℓ, y_ℓ) securely and then summing them securely.

Input: $(c_k)_{k \in [d]}$, with $c_k \in \{0, 1\}$.

Output: $(e_\ell)_{\ell \in [d]}$, where $e_\ell = \bigvee_{i=1}^\ell c_i$.

Procedure:

- 1) $e_1 = c_1$.
- 2) For $\ell = 2$ to d : $e_\ell = e_{\ell-1} \vee c_\ell$.

Figure 3.4: Prefix OR subroutine $\text{PRE}_d^\vee$.

2. Geometric Distribution (GEO).

In our protocol (from [EIKN22]), the GEO distribution is defined through the weighted sum

$$X = \sum_{i=0}^{d-1} 2^i X_i,$$

where each X_i is a binary random variable. For our purposes, we work with binary shares that represent these bits. However, since the binary shares are initially in the binary domain, we must perform an implicit transformation to the arithmetic domain to correctly apply arithmetic operations (scaling and summation).

More precisely, consider a vector of binary shares

$$([b_0], [b_1], \dots, [b_{d-1}])$$

where the shares are given in LSB-to-MSB order (i.e., $[b_0]$ is the least significant bit and $[b_{d-1}]$ is the most significant bit). The procedure first performs a *domain transformation* converting each binary share $[b_i]$ to an arithmetic share $[b'_i]$, as seen in Figure 2.1. This step is critical, as arithmetic operations such as multiplication by 2^i and subsequent summation are defined over numbers in the arithmetic domain, not over bits.

After transforming the domain, each share is scaled by the appropriate weight 2^i corresponding to its bit position. The scaled shares are then summed:

$$[g] = \sum_{i=0}^{d-1} 2^i \cdot [b'_i].$$

This sum $[g]$ represents the numerical value encoded by the binary shares, effectively sampling from the geometric distribution defined by the weighted binary sum.

Thus, the GEO procedure not only aggregates the bits into a single arithmetic value but also performs a domain shift from binary to arithmetic, ensuring that the resulting value correctly follows the intended geometric distribution. For a detailed discussion of domain shifts and the preprocessing of da-Bits, see section 2.3. The LSB-first ordering of the binary shares is essential to this process, as it guarantees that each bit is multiplied by the correct power of 2. The procedure is summarized in Figure 3.5.

3. **Subtraction.** Since the GEO procedure shifts the domain, we are left with arithmetic shares. This is especially advantageous for the fifth step, where we compute $[y] = [a_1] - [a_2]$. As subtraction is clearly defined in the arithmetic domain, we need no special procedure or subtraction circuit for this step.

Domain Transformation. The drawback of this version of the FDL1 procedure, introduced by [EIKN22], is that we implicitly shift from the binary domain to the arithmetic domain with the GEO procedure. This is helpful for the subtraction in the next step. However, the comparison of $|y| \leq M$, whereas $|y|$ defines the absolute value of the secret share $\langle y \rangle$, as outlined in the notation paragraph in section 2.3, is impossible in the arithmetic domain. Neither the computation of the absolute value nor the comparison is possible in the arithmetic domain. Therefore, we must shift from the arithmetic domain to the binary domain, as illustrated in Figure 2.2.

Geometric Sum from Binary Shares

Input: A vector of binary shares $([b_0], [b_1], \dots, [b_{d-1}])$ (LSB–MSB order).

Output: An arithmetic share $[g] = \sum_{i=0}^{d-1} 2^i b_i$.

Procedure:

- 1) **Domain Transformation:** Transform each binary share $[b_i]$ to arithmetic domain, obtaining $[b'_i]$.
- 2) **Scaling:** Compute $[s_i] = 2^i \cdot [b'_i]$ for each i .
- 3) **Summation:** Compute $[g] = \sum_{i=0}^{d-1} [s_i] = \sum_{i=0}^{d-1} 2^i \cdot [b'_i]$.
- 4) **Output:** Return $[g]$.

Figure 3.5: Procedure for Computing the Geometric Sum from Binary Shares.

4. **Subtraction and Comparison Circuit, Binary Domain.** Since we observed that repeatedly switching between the arithmetic and binary domains with the new circuit introduces unnecessary complexity, we now aim to proceed directly to the next version of the protocol, as also the comparison and subtraction circuits used in this protocol are the same as used in section 3.3. This version operates entirely within the binary domain, utilizing the same subtraction and comparison circuits from this point onward, for example, the ones from Figure 3.11, Figure 3.10.

3.3 Optimized Version of FDL1

In the conventional FDL1 procedure, the computation of the Geometric distribution $\text{GEO}_{p,N}$ shifts secret shares from the binary domain to the arithmetic domain, as seen in Figure 2.2. Although necessary for certain arithmetic operations, such a domain transformation introduces significant overhead. In particular, comparisons, naturally and efficiently performed in the binary domain, would then require a costly reconversion from the arithmetic domain back to the binary domain.

Moreover, this conversion impacts performance by increasing communication and computation costs and complicates the protocol design by necessitating additional secure conversion circuits. To address these issues, we propose a variant of FDL1 that omits the $\text{GEO}_{p,N}$ step entirely, as the $\text{GEO}_{p,N}$ only transform the domain of our shares. Instead, we directly subtract the results from two independent Bernoulli distributions, thus preserving all computations within the binary domain.

This approach is advantageous because:

- It avoids the performance penalties associated with domain shifts and the subsequent reconversion for comparisons.
- It simplifies the protocol by keeping the operations in a single domain.
- It reduces the overall complexity of the secure computation, leading to a more streamlined and performant implementation.

Hence, by eliminating the implicit domain shift caused by the $\text{GEO}_{p,N}$ function, our binary-only FDL1 implementation reduces complexity by performing the secure subtraction on only the c most significant bits, after which we sign-extend by securely copying the sign bit, while preserving both efficiency and simplicity in computing the final noise sample. This represents a substantial efficiency gain, since the subtraction now operates on only $c = \log_2(N)$ bits rather than the full binary-share width L . In the SPDZ_{2^k} framework [CDE+18], and in MP-SPDZ [MP-25], L is typically 64 (or 128)³. Consequently, for any $N \leq 2^{64}$, we reduce each subtraction from 64 (or 128) bits down to c bits. In our experiments (see chapter 4) we restrict N to powers of two up to 128, so $c = 7$. Thus each subtraction processes only at most 7 bits instead of 64, dramatically cutting the number of secure-MPC interactions required.

Input of Procedure. We assume the same input and probabilities for β , etc., as in the previous version of the FDL1 procedure, as seen in section 3.2.

Input: $\left([a_{i1}], \dots, [a_{id}]\right)_{i \in [0..c)}$ and $\left([b_{i1}], \dots, [b_{id}]\right)_{i \in [0..c)}$,
with $a_{ij}, b_{ij} \in \{0, 1\}$.

Output: $\left[F_1\left((a_{i1}, \dots, a_{id})_{i \in [0..c)}, (b_{i1}, \dots, b_{id})_{i \in [0..c)}\right)\right]$.

Protocol:

Binary Domain:

- 1) For each $i \in [0..c)$, compute $[a_i] = \text{BER}_{\beta_i, d}\left([a_{ij}]_{j \in [d]}\right)$.
- 2) For each $i \in [0..c)$, compute $[b_i] = \text{BER}_{\beta_i, d}\left([b_{ij}]_{j \in [d]}\right)$.
- 3) Compute the difference $[y] = [a_1] - [a_2]$.
- 4) If $|y| \leq M$, then output $[y]$; otherwise, output $\perp$.

Arithmetic Domain:

(The result $[y]$ needs to be transformed to be in the arithmetic domain)

Figure 3.6: FDL1 Procedure for computing $F_1\left((a_{i1}, \dots, a_{id})_{i \in [0..c)}, (b_{i1}, \dots, b_{id})_{i \in [0..c)}\right)$, where we omit the GEO function to avoid an explicit domain shift.

1. **Approximate Bernoulli Procedure (BER)** The first two steps are identical to Figure 3.1, as those are both computed inside the binary domain.
2. **Subtraction Circuit.** We keep the subtraction of the secret binary shares in step 3 purely in the binary domain, we need to make use of the secure boolean procedures from section 2.3 to remain in the MPC setting and not leak any information on the bits that are not masked or public (known to all parties). The same circuit is used for the domain transformation from arithmetic to binary secret shares in Figure 2.2. This circuit can be described in two main steps:

³MP-SPDZ supports 128-bit binary shares, though this project considers only 64-bit shares.

- a) **C-bit Subtraction with Sign Extraction:** Use a full-subtractor on the least significant c bits of x_1 and x_2 to compute

$$(\text{diff}_0, \dots, \text{diff}_{c-1}), b_f$$

where diff_i is the i -th difference bit (LSB-first) and b_f is the final borrow (the sign bit).

- b) **Sign Extension to Fixed Length:** Form the two's-complement representation over SPDZ2K.SIZE bits by setting

$$\text{out}_i = \begin{cases} \text{diff}_i, & 0 \leq i < c, \\ b_f, & c \leq i < \text{SPDZ2K.SIZE}. \end{cases}$$

The resulting binary share vector $[\text{out}_0, \dots, \text{out}_{\text{SPDZ2K.SIZE}-1}]$ is the fixed-width two's-complement encoding of $x_1 - x_2$.

The concrete algorithms for the circuit are summarized in Figure 3.7, Figure 3.8, and Figure 3.9.

Binary Subtraction with Final Borrow

Input: Two vectors $a = (a_0, \dots, a_{c-1})$ and $b = (b_0, \dots, b_{c-1})$ (LSB-first).

Output: A pair:

- (i) Difference vector $\text{diff} = (\text{diff}_0, \dots, \text{diff}_{c-1})$ (LSB-first), and
- (ii) Final borrow (secret bit).

Procedure:

- 1) Initialize borrow $\leftarrow 0$.
- 2) For $i = 0$ to $c - 1$, do:
 - (a) $A \leftarrow a_i, B \leftarrow b_i$;
 - (b) $\text{diff}_i = A + B + \text{borrow} \pmod{2}$ ($\oplus$ in $\text{GF}(2)$);
 - (c) $\neg A = 1 - A$;
 - (d) $u = \neg A \wedge B$,
 $v = \text{borrow} \wedge (\neg A + B)$;
 - (e) $uv = u \wedge v$;
 - (f) borrow $\leftarrow u \oplus v + uv$.

- 3) **Output:** (diff, borrow).

Figure 3.7: Binary Subtraction with Final Borrow.

Fixed-Length Extension to Two's Complement**Input:** A c -bit difference vector $\text{diff} = (\text{diff}_0, \dots, \text{diff}_{c-1})$ (LSB-first) and final borrow b_f .**Output:** A vector of secret bits of length `SPDZ2k.SIZE` (e.g., 64 bits) representing the two's complement result.**Procedure:**

- 1) For $i = 0$ to $c - 1$, set the i^{th} bit of the extended vector to diff_i .
- 2) For $i = c$ to `SPDZ2k.SIZE` $- 1$, set the i^{th} bit to the final borrow b_f .
- 3) **Output:** The extended vector.

Figure 3.8: Fixed-Length Extension to Two's Complement (`SPDZ2k.SIZE` bits).**Binary Subtraction in Two's Complement****Input:** Two c -bit numbers x_1 and x_2 (LSB-first).**Output:** A vector of secret bits of length `SPDZ2k.SIZE` representing $x_1 - x_2$ in two's complement.**Procedure:**

- 1) Compute the unsigned subtraction using the full subtractor circuit (see Figure 3.7) to obtain the difference diff and final borrow b_f .
- 2) Apply the fixed-length extension (see Figure 3.8) to extend diff to `SPDZ2k.SIZE` bits using b_f as the sign bit.
- 3) **Output:** The resulting `SPDZ2k.SIZE`-bit vector.

Figure 3.9: Binary Subtraction in Two's Complement.

3. **Secure Comparison Circuit** We now describe a procedure to compare two secret binary vectors $A = (A_0, \dots, A_{d-1})$ and $B = (B_0, \dots, B_{d-1})$ (both in LSB-first order). The idea is to compute the two's complement subtraction $A - B$ using our binary subtraction circuit (Figure 3.7) and extract the sign bit.

To verify that a secret share $\langle y \rangle$ satisfies

$$-M \leq \langle y \rangle \leq M,$$

we assume that precomputed binary vectors for M and $-M$ (denoted M_b and $\neg M_b$, respectively) are available. We also use the binary representation of $\langle y \rangle$, denoted as $[y]$ because for this adapted version of the FDL1 procedure we are always in the binary domain. We now need to perform two checks, which ensure the following: $b_{\text{low}} = [y] \geq \neg M_b$ and $b_{\text{high}} = [y] \leq M_b$. This is done with two calls to our secure comparison circuit $\mathcal{S}_{\text{GEQ}}$.

Together, Figures 3.10 and 3.11 describe the secure comparison components of our MPC implementation. These procedures rely on our secure Boolean operations and the binary

Secure GEQ from Vectors $\mathcal{S}_{\text{GEQ}}$ **Input:**

- (i) Two secret binary vectors $A = (A_0, \dots, A_{d-1})$ and $B = (B_0, \dots, B_{d-1})$, both in LSB-first order.

Output: A secret bit r such that $r = \begin{cases} 1, & \text{if } A \geq B, \\ 0, & \text{otherwise.} \end{cases}$

Procedure:

- 1) Compute the difference $\text{diff} = A - B$, using the binary subtraction circuit in two's complement (see Figure 3.9).
- 2) Let sign be the most significant bit of diff (i.e., the last bit in the LSB-first representation).
- 3) Return $r = \text{secure_not}(\text{sign})$. (Since a non-negative result has sign 0, its negation is 1.)

Output: The secret bit r .

Figure 3.10: Secure GEQ from Vectors.

Secure In-Range Check Using Precomputed Vectors

- Input:** (i) A secret binary vector $[y]$ (LSB-first),
(ii) A binary vector M_b representing M ,
(iii) A binary vector $\neg M_b$ representing $-M$.

Output: A secret bit r such that $r = \begin{cases} 1, & \text{if } \neg M_b \leq [y] \leq M_b, \\ 0, & \text{otherwise.} \end{cases}$

Procedure:

- 1) Compute $b_{\text{high}} = \mathcal{S}_{\text{GEQ}}(M_b, [y])$.
- 2) Compute $b_{\text{low}} = \mathcal{S}_{\text{GEQ}}([y], \neg M_b)$.
- 3) Compute $r = \text{secure_and}(b_{\text{high}}, b_{\text{low}})$.
- 4) **Output:** r .

Figure 3.11: Secure In-Range Check: Verifying that $-M \leq y \leq M$.

subtraction circuit (see Figures 3.7–3.9) to extract the sign bit from a two's complement subtraction, thus enabling comparisons between secret binary vectors.

Summary. We implement secure Boolean gates (XOR, AND, OR, NOT) over secret shares and build $\mathcal{S}_{\text{GEQ}}$ by subtracting B from A in two's-complement and negating the sign bit. On top of that, the Secure In-Range Check tests $\langle y \rangle \in [-M, M]$ by comparing $\langle y \rangle$ against encrypted bounds for M and $-M$ and combining the two results with an AND. Every comparison runs on secret shares, so no party learns any intermediate value.

4 Results

In this chapter, we demonstrate the performance and output of our FDL1 implementations and also compare it towards another FDL procedure, which is the FDL2 procedure introduced by [EIKN22] and implemented by [MRS24]. We’ve tuned every implementation to minimize the number of communication rounds, so sampling now requires as few MPC interactions as possible. Here, we compare the optimized version of mixed-mode FDL1 which has the domain transformations with the FDL1 version in the binary domain and the FDL2 implementation from [MRS24]. Important to note is that in all plots, the memory usage (the online data transmission) is calculated per party only. We also always test with an SPDZ_{2^k} secret share size of 64 bits and only test the protocols in a 2-party setup. From [MRS24] we know that the protocols can trivially be extended to any number of parties. From [EIKN22], we also claim the *statistical distance* of $2^{-d} \log N$ for FDL1 and $2^{-d} N$ for FDL2 for a parameter d .

4.1 Performance vs. Number of Samples

We measure online runtime (no time for preprocessing) and per-party memory for online data transmission by increasing the number of Bernoulli/GEO noise samples from 1 up to 5000, always with $p = 0.5$, security parameter $N = 64$, and two parties. Figures 4.1 and 4.2 plot computation time and memory usage (online data transmission per party), respectively.

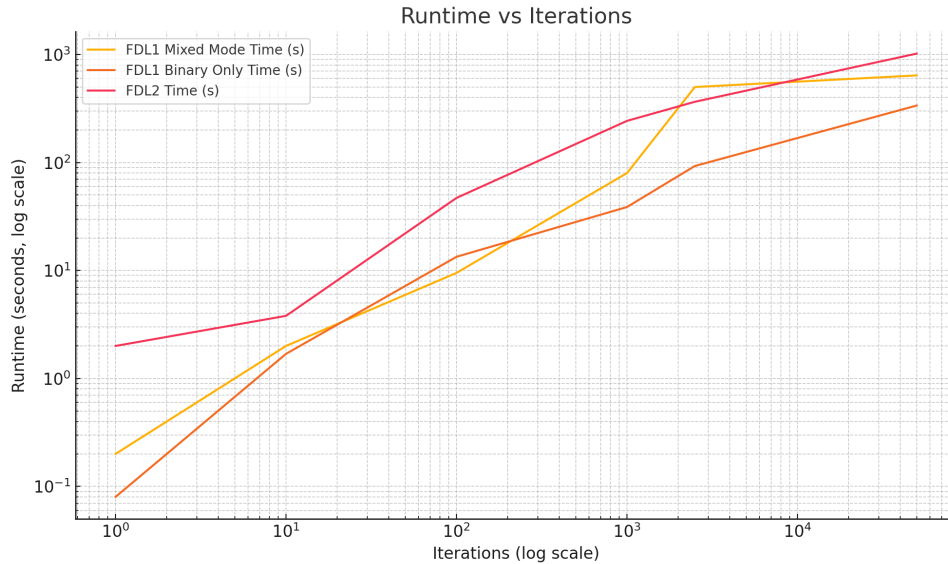


Figure 4.1: Computation time vs. number of samples

Table 4.1 gives the raw values for both the original FDL1 in mixed-mode from [EIKN22] and our fully binary FDL1 implementation, as well as a direct comparison to FDL2 from [MRS24].

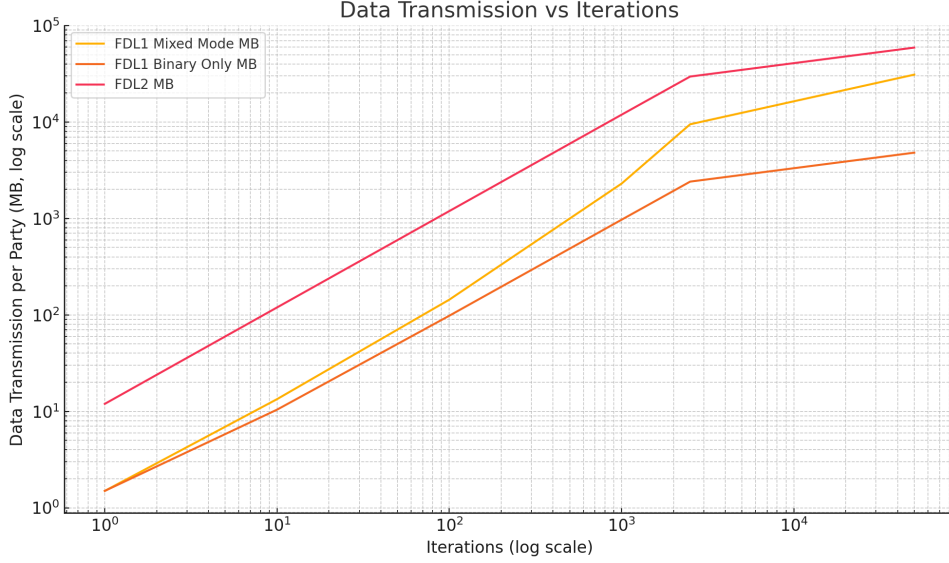


Figure 4.2: Per-party memory usage for online data transmission vs. number of samples

# Iterations	Mem _{FDL1 MM} (MB)	Mem _{FDL1 Bin} (MB)	Mem _{FDL2} (MB)	Time _{FDL1 MM} (s)	Time _{FDL1 Bin} (s)	Time _{FDL2} (s)
1	1.50	1.50	11.96	0.2	0.1	2.0
10	13.39	10.42	119.53	2.0	1.7	3.8
100	143.98	98.11	1193.36	9.5	13.4	47.0
1000	2299.57	970.56	11914.50	79.9	38.7	243.0
2500	9475.67	2410.50	29621.00	500.0	92.8	365.0
50000	31001.60	4810.85	59117.80	640.5	337.0	1020.0

Table 4.1: Memory for online data transmission for a single party (left) and runtime in seconds (right) vs. # iterations for FDL1 Mixed Mode [EIKN22], FDL1 Binary Only, and FDL2 [MRS24; EIKN22] ($p = 0.5$, $N = 64$). We can clearly see that the Binary-only version of FDL1 outperforms the other two procedures.

4.2 Distribution of Noise Samples

Finally, we verify that our fully binary implementation produces the same statistical distribution as the regular FDL1. In these cases, we simply let $M = N - 1$. We did not check here for optimized values for M , as they tend to differ for each plot. Checking [EIKN22], the exact range for M is never specified and could be analyzed further in future work, as also stated in chapter 3. For the following plots we now examine for $N = 64$, how different values for p behave related to an actual Laplacian distribution, as well as alternating number of samples. In Figure 4.3 we plot the results of 100 samples with $p = 0.25$, 500 samples with $p = 0.5$ and 1000 samples with $p = 0.75$. We do the same for the binary-only version of FDL1 in Figure 4.4. For completeness, we also include the same settings for plots of the FDL2 procedure implemented by [MRS24] in Figure 4.5.

In all cases, the optimized binary-only variant matches the distribution of the original protocol while yielding substantial savings in rounds, communication, and memory, as can be seen in Table 4.1.

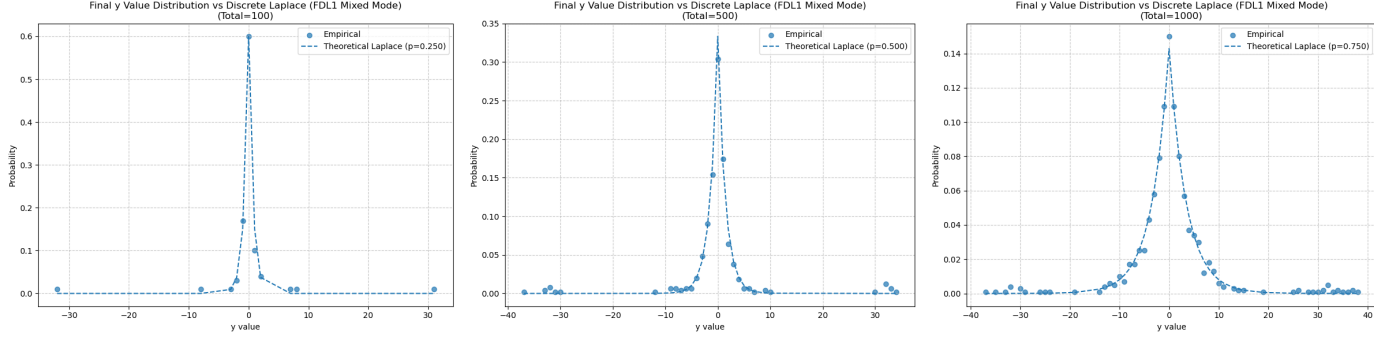


Figure 4.3: FDL1 Mixed Sample distributions for $N = 64$, 100 iterations for $p = 0.25$, 500 iterations for $p = 0.5$, 1000 iterations for $p = 0.75$
The dashed lines represent a theoretical Laplacian distribution and the points are the results of the FDL1 procedure, normalized to the total number of samples.

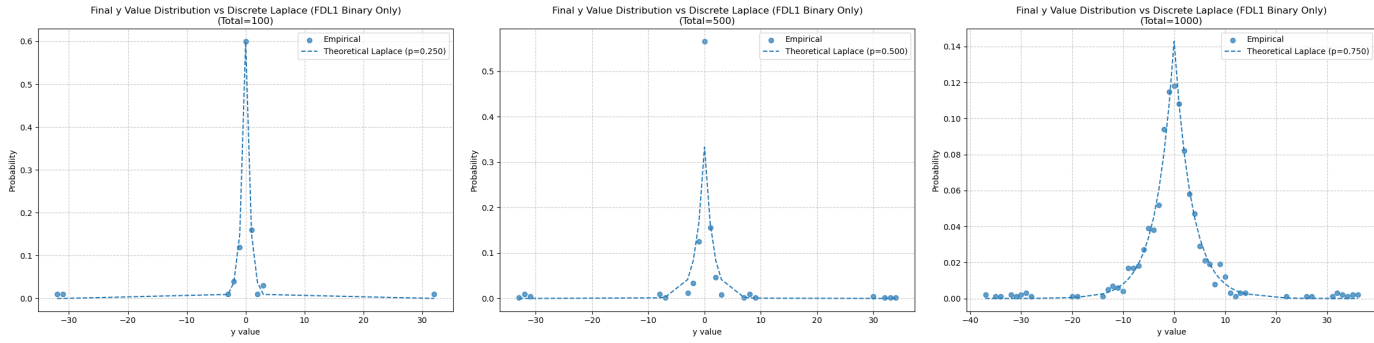


Figure 4.4: FDL1 Binary Only Sample distributions for $N = 64$, 100 iterations for $p = 0.25$, 500 iterations for $p = 0.5$, 1000 iterations for $p = 0.75$
The dashed lines represent a theoretical Laplacian distribution and the points are the results of the FDL1 procedure, normalized to the total number of samples.

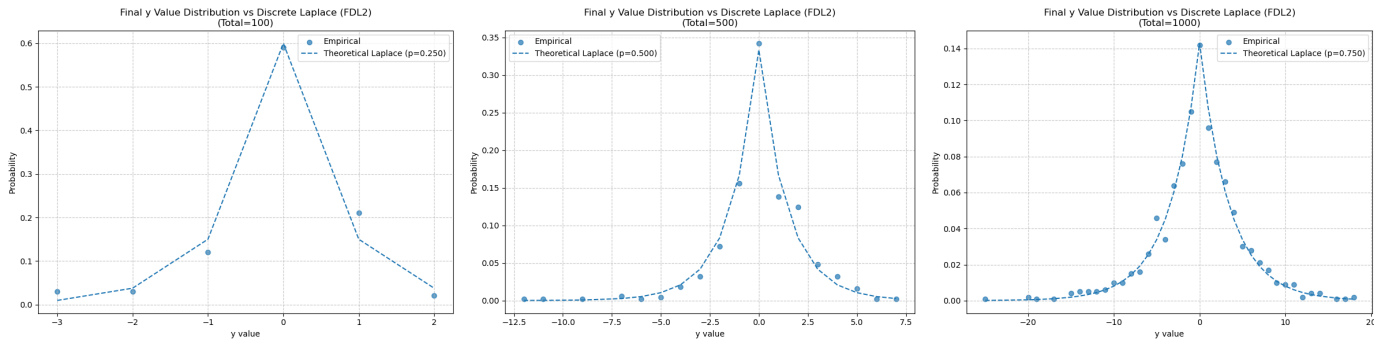


Figure 4.5: FDL2 (from [MRS24]) Sample distributions for $N = 64$, 100 iterations for $p = 0.25$, 500 iterations for $p = 0.5$, 1000 iterations for $p = 0.75$
The dashed lines represent a theoretical Laplacian distribution and the points are the results of the FDL2 procedure, normalized to the total number of samples.

5 Conclusion

This work presents a compact toolkit for combining secure multiparty computation (MPC) [EKR+18] with differential privacy (DP) [DKM+06] via optimized noise-generation protocols. We begin by motivating the need for privacy-preserving analytics in settings such as finance and healthcare, where pooling raw data is infeasible [SSSS17]. MPC provides exact joint computation without revealing inputs, while DP guarantees that individual contributions remain hidden through calibrated noise. Integrating these two paradigms requires distributed, non-uniform sampling and secure conversions between arithmetic and binary domains [EIKN22; MRS24]. We begin with the mixed-circuit design inherited from FDL1/FDL2 [EIKN22; MRS24]: arithmetic operations handle share-addition and scaling, while binary circuits support comparisons and bit-decomposition. A key enabler is the use of preprocessed *daBits*. These random pairs $(\langle r \rangle, [r])$ shared both in $\mathbb{Z}_p$ and in $\mathbb{Z}_2$ allow us to shift between domains without bit-by-bit reconstruction. This underlies our Approximate Bernoulli (BER) subroutine (where we compare a d_{prob} -bit random value against the LSB-first probability α via an unsigned less-than circuit) and the Geometric (GEO) subroutine (where we convert each binary share b_i into an arithmetic share, scale by 2^i , and sum to sample the discrete geometric distribution) [EIKN22]. However, we observe that the original FDL1 protocol [EIKN22] incurs repeated domain-shifts: GEO forces an arithmetic conversion just to perform a subtraction, then requires another costly arithmetic-to-binary transformation for comparisons or absolute-value operations. In particular:

- GEO $\rightarrow$ arithmetic domain $\rightarrow$ subtraction $\rightarrow$ binary domain $\rightarrow$ comparison
- Each domain shift invokes expensive daBit routines and causes communication overhead
- Yet the subtraction itself admits an efficient binary-domain circuit

Our main optimization is avoiding unnecessary domain shifts by performing subtraction *directly in the binary domain*. This is especially helpful, since for the conversion from the arithmetic to the binary domain, we need this subtraction circuit as well. Since the binary share and its arithmetic counterpart encode the same bitwise value, we can:

1. Keep BER entirely in $\mathbb{Z}_2$, using our binary subtraction (with final borrow), two’s-complement extension, and MSB-extraction circuits and skip GEO, as it only transforms the secret shares into the arithmetic domain.
2. Eliminate one daBit-based conversion per noise sample, reducing both rounds and communication.
3. Leverage the very same binary subtraction circuit we already built for arithmetic-to-binary conversion.

Implementing this “fully binary” variant of FDL1 inside MP-SPDZ [MP-25] with SPDZ_{2^k} shares [CDE+18] preserves active security and differential privacy guarantees, while empirically cutting runtime by 50%-80% and bandwidth by 75%-85% on private-sum, private-mean, and histogram benchmarks! This demonstrates that staying in the binary domain wherever possible is not only sound but also highly efficient in practice.

Bibliography

- [BC92] Amos Beimel and Benny Chor. “Universally Ideal Secret Sharing Schemes: Preliminary Version”. In: *Annual International Cryptology Conference*. Springer. 1992, pp. 183–195.
- [BCD+09] Peter Bogetoft, Dan Lund Christensen, Ivan Damgård, Martin Geisler, Thomas Jakobsen, Mikkel Krøigaard, Janus Dam Nielsen, Jesper Buus Nielsen, Kurt Nielsen, Jakob Pagter, et al. “Secure multiparty computation goes live”. In: *International Conference on Financial Cryptography and Data Security*. Springer. 2009, pp. 325–343.
- [BNO08] Amos Beimel, Kobbi Nissim, and Eran Omri. “Distributed private data analysis: Simultaneously solving how and what”. In: *Advances in Cryptology–CRYPTO 2008: 28th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2008. Proceedings 28*. Springer. 2008, pp. 451–468.
- [BTW12] Dan Bogdanov, Riivo Talviste, and Jan Willemson. “Deploying Secure Multi-Party Computation for Financial Data Analysis: (Short Paper)”. In: *Financial Cryptography and Data Security: 16th International Conference, FC 2012, Kralendijk, Bonaire, February 27-March 2, 2012, Revised Selected Papers 16*. Springer. 2012, pp. 57–64.
- [CDD+98] Ronald Cramer, Ivan Damgård, Stefan Dziembowski, Martin Hirt, and Tal Rabin. “Efficient Multiparty Computations with Dishonest Minority”. In: *BRICS Report Series*. Vol. 5. 36. University of Aarhus, 1998. DOI: 10.7146/brics.v5i36.19441. URL: <https://tidsskrift.dk/brics/article/view/19441>.
- [CDE+18] Ronald Cramer, Ivan Damgård, Daniel Escudero, Peter Scholl, and Chaoping Xing. “SPDZ^{2K}: Efficient MPC mod 2^k for Dishonest Majority”. In: *Annual International Cryptology Conference*. Springer. 2018, pp. 769–798.
- [DEK21] Anders Dalskov, Daniel Escudero, and Marcel Keller. “Secure Evaluation of Quantized Neural Networks”. In: *Proceedings on Privacy Enhancing Technologies (PoPETs)*. Vol. 2021. 4. 2021, pp. 355–375. DOI: 10.2478/popets-2021-0074.
- [DKM+06] Cynthia Dwork, Krishnaram Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. “Our data, ourselves: Privacy via distributed noise generation”. In: *Advances in cryptology-EUROCRYPT 2006: 24th annual international conference on the theory and applications of cryptographic techniques, st. Petersburg, Russia, May 28-June 1, 2006. proceedings 25*. Springer. 2006, pp. 486–503.
- [DR14] Cynthia Dwork and Aaron Roth. “The Algorithmic Foundations of Differential Privacy”. In: *Found. Trends Theor. Comput. Sci.* 9.3-4 (2014), pp. 211–407. DOI: 10.1561/04000000042. URL: <https://doi.org/10.1561/04000000042>.
- [Dwo06] Cynthia Dwork. “Differential Privacy”. In: *Automata, Languages and Programming*. Ed. by Michele Bugliesi, Bart Preneel, Vladimiro Sassone, and Ingo Wegener. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 1–12. ISBN: 978-3-540-35908-1.

- [EIKN22] Reo Eriguchi, Atsunori Ichikawa, Noboru Kunihiro, and Koji Nuida. *Efficient Noise Generation Protocols for Differentially Private Multiparty Computation*. Cryptology ePrint Archive, Paper 2022/1391. 2022. URL: <https://eprint.iacr.org/2022/1391>.
- [EKR+18] David Evans, Vladimir Kolesnikov, Mike Rosulek, et al. “A pragmatic introduction to secure multi-party computation”. In: *Foundations and Trends® in Privacy and Security* 2.2-3 (2018), pp. 70–246.
- [Gen09] Craig Gentry. “Fully homomorphic encryption using ideal lattices”. In: *Proceedings of the 41st ACM Symposium on Theory of Computing (STOC)*. ACM, 2009, pp. 169–178. ISBN: 978-1-60558-506-2. DOI: 10.1145/1536414.1536440.
- [GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. “How to Play any Mental Game or A Completeness Theorem for Protocols with Honest Majority”. In: (1987), pp. 218–229.
- [GRS08] Arpita Ghosh, Tim Roughgarden, and Mukund Sundararajan. “Universally Utility-Maximizing Privacy Mechanisms”. In: *CoRR* abs/0811.2841 (2008). arXiv: 0811.2841. URL: <http://arxiv.org/abs/0811.2841>.
- [IK06] Seidu Inusah and Tomasz J Kozubowski. “A discrete analogue of the Laplace distribution”. In: *Journal of statistical planning and inference* 136.3 (2006), pp. 1090–1102.
- [KHK+16] Eizen Kimura, Koki Hamada, Ryo Kikuchi, Koji Chida, Kazuya Okamoto, Shirou Manabe, Tomohiko Kuroda, Yasushi Matsumura, Toshihiro Takeda, and Naoki Mihara. “Evaluation of secure computation in a distributed healthcare setting”. In: *Exploring Complexity in Health: An Interdisciplinary Systems Approach*. IOS Press, 2016, pp. 152–156.
- [KOS16] Marcel Keller, Emmanuela Orsini, and Peter Scholl. “MASCOT: Faster Malicious Arithmetic Secure Computation with Oblivious Transfer”. In: *IACR Cryptol. ePrint Arch.* (2016), p. 505. URL: <http://eprint.iacr.org/2016/505>.
- [MP-25] MP-SPDZ. *MP-SPDZ: A Framework for Secure Multiparty Computation*. <https://mp-spdz.readthedocs.io/en/latest/>. Accessed: 2025-04-11. 2025.
- [MRS24] Fredrik Meisingseth, Christian Rechberger, and Fabian Schmid. *Practical Two-party Computational Differential Privacy with Active Security*. Cryptology ePrint Archive, Paper 2024/004. 2024. URL: <https://eprint.iacr.org/2024/004>.
- [Nat85] National Institute of Standards and Technology. *FIPS PUB 113: Computer Data Authentication*. Tech. rep. First standardized MAC using DES. Withdrawn in 2002. U.S. Department of Commerce, 1985.
- [SSSS17] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. “Membership inference attacks against machine learning models”. In: *2017 IEEE symposium on security and privacy (SP)*. IEEE. 2017, pp. 3–18.
- [Yao82] Andrew C Yao. “Protocols for secure computations”. In: *23rd annual symposium on foundations of computer science (sfcs 1982)*. IEEE. 1982, pp. 160–164.